



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4.1
по курсу «Компьютерные сети»
«Разработка SSH-сервера и SSH-клиента»

Студент группы ИУ9-32Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Реализовать ssh сервер на языке GO. Должны работать следующие функции:

- авторизация клиента на ssh сервере;
- создание директории на удаленном сервере;
- удаление директории на удаленном сервере;
- вывод содержимого директории;
- перемещение файлов из одной директории в другую;
- удаление файла по имени;
- вызов внешних приложений, например ping.

Требования: SSH-клиент должен поддерживать следующие функции:

- авторизация клиента на SSH-сервере;
- создание директории на удаленном SSH-сервере;
- удаление директории на удаленном SSH-сервере;
- вывод содержимого директории;
- перемещение файлов из одной директории в другую;
- удаление файла по имени;
- вызов внешних приложений, например ping.

2 Результаты

Исходный код программы представлен в листинге 1–2.

Листинг 1: Реализация файла server.go

```
1 package main
2
3 import (
4     "fmt"
5     "io"
6     "io/ioutil"
7     "log"
8     "os"
9     "os/exec"
10    "github.com/gliderlabs/ssh"
11 )
12
13 // Function to execute system commands
14 func runCommand(cmd string, args ...string) string {
```

```

15         out, err := exec.Command(cmd, args...).CombinedOutput()
16         if err != nil {
17             return fmt.Sprintf("Error: %s\n%s", err, string(out))
18         }
19         return string(out)
20     }
21
22     func main() {
23         ssh.Handle(func(s ssh.Session) {
24             user := s.User()
25             fmt.Printf("User %s connected to the server\n", user) // Log
                connection
26
27             commands := s.Command()
28             if len(commands) < 1 {
29                 io.WriteString(s, "No command specified\n")
30                 return
31             }
32
33             fmt.Printf("User %s executes command: %s\n", user, commands) //
                Log command
34
35             switch commands[0] {
36             case "mkdir":
37                 if len(commands) < 2 {
38                     io.WriteString(s, "Specify a directory
                to create\n")
39                     return
40                 }
41                 err := os.Mkdir(commands[1], 0755)
42                 if err != nil {
43                     io.WriteString(s, fmt.Sprintf("Error
                creating directory: %s\n", err))
44                 } else {
45                     io.WriteString(s, "Directory created\n")
46                 }
47
48             case "rmdir":
49                 if len(commands) < 2 {
50                     io.WriteString(s, "Specify a directory
                to delete\n")
51                     return
52                 }
53                 err := os.Remove(commands[1])
54                 if err != nil {

```

```

55         io.WriteString(s, fmt.Sprintf("Error
deleting directory: %s\n", err))
56     } else {
57         io.WriteString(s, "Directory deleted\n")
58     }
59
60     case "list":
61         if len(commands) < 2 {
62             commands = append(commands, ".")
63         }
64         files, err := ioutil.ReadDir(commands[1])
65         if err != nil {
66             io.WriteString(s, fmt.Sprintf("Error
reading directory: %s\n", err))
67             return
68         }
69         for _, file := range files {
70             io.WriteString(s, file.Name()+"\n")
71         }
72
73     case "move":
74         if len(commands) < 3 {
75             io.WriteString(s, "Specify source and
destination\n")
76             return
77         }
78         err := os.Rename(commands[1], commands[2])
79         if err != nil {
80             io.WriteString(s, fmt.Sprintf("Error
moving file: %s\n", err))
81         } else {
82             io.WriteString(s, "File moved\n")
83         }
84
85     case "remove":
86         if len(commands) < 2 {
87             io.WriteString(s, "Specify a file to
delete\n")
88             return
89         }
90         err := os.Remove(commands[1])
91         if err != nil {
92             io.WriteString(s, fmt.Sprintf("Error
deleting file: %s\n", err))
93         } else {
94             io.WriteString(s, "File deleted\n")

```

```

95         }
96
97         case "ping":
98             if len(commands) < 2 {
99                 io.WriteString(s, "Usage: ping <hostname
>\n")
100                 return
101             }
102             out := runCommand("ping", commands[1], "-c", "4"
)
103             io.WriteString(s, string(out))
104             log.Printf(user, "ping", commands[1], "success")
105         default:
106             io.WriteString(s, "Unknown command\n")
107     }
108 })
109
110     log.Println("Starting SSH server on port 5410...")
111     err := ssh.ListenAndServe(":5410", nil, ssh.PasswordAuth(func(
ctx ssh.Context, pass string) bool {
112         return pass == "password"
113     })))
114     if err != nil {
115         log.Fatal("Error starting SSH server:", err)
116     }
117 }

```

Листинг 2: Реализация файла client.go

```

1 package main
2
3 import (
4     "bufio"
5     "fmt"
6     "golang.org/x/crypto/ssh"
7     "log"
8     "os"
9     //"strings"
10 )
11
12 // Function to connect to the SSH server
13 func connectToSSH(user, password, host string, port int) (*ssh.Client,
error) {
14     config := &ssh.ClientConfig{
15         User: user,
16         Auth: []ssh.AuthMethod{
17             ssh.Password(password),

```

```

18         },
19         HostKeyCallback: ssh.InsecureIgnoreHostKey(),
20     }
21
22     address := fmt.Sprintf("%s:%d", host, port)
23     client, err := ssh.Dial("tcp", address, config)
24     if err != nil {
25         return nil, fmt.Errorf("failed to connect to SSH server:
26         %w", err)
27     }
28     return client, nil
29 }
30 // Function to run a command on the server
31 func runCommand(client *ssh.Client, command string) (string, error) {
32     session, err := client.NewSession()
33     if err != nil {
34         return "", fmt.Errorf("failed to create session: %w",
35         err)
36     }
37     defer session.Close()
38
39     output, err := session.CombinedOutput(command)
40     if err != nil {
41         return "", fmt.Errorf("error executing command: %w", err
42     )
43     }
44
45     return string(output), nil
46 }
47
48 // Interface for reading commands from the user
49 func interactiveShell(client *ssh.Client) {
50     scanner := bufio.NewScanner(os.Stdin)
51     fmt.Println("Connected to SSH server. Enter commands (e.g.,
52     mkdir, removedir, list, move, remove, ping). Type 'exit' to
53     disconnect.")
54
55     for {
56         fmt.Print("Enter command: ")
57         scanner.Scan()
58         command := scanner.Text()
59         if command == "exit" {
60             fmt.Println("Disconnecting from server...")
61             break
62         }
63     }
64 }

```

```

59
60         output, err := runCommand(client, command)
61         if err != nil {
62             log.Printf("Command execution error: %v\n", err)
63         } else {
64             fmt.Println("Result:\n", output)
65         }
66     }
67 }
68
69 func main() {
70     // Connection settings
71     user := "OLYA"           // replace with username
72     password := "password"   // replace with password
73     host := "185.102.139.168" // SSH server address
74     port := 5410             // SSH server port
75
76     // Connect to SSH server
77     client, err := connectToSSH(user, password, host, port)
78     if err != nil {
79         log.Fatalf("Failed to connect to server: %v", err)
80     }
81     defer client.Close()
82
83     // Start interactive mode
84     interactiveShell(client)
85 }

```

Результат запуска представлен на рисунках 1– 3.

```

root@net3:~/Ivenkova/lab4.1# go run server.go
2024/11/03 18:52:07 Запуск SSH-сервера на порту 5410...
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [list]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [mkdir testtest]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [list]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [removedir test1]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [list]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [move abc1.txt testtest/abc.txt]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [list]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [list testtest]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [move testtest/abc.txt abc2.txt]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [list]
Пользователь OLYA подключился к серверу
Пользователь OLYA выполняет команду: [ping google.com]
2024/11/03 18:55:11 OLYA%(EXTRA string=ping, string=google.com, string=success)
^Csignal: interrupt
root@net3:~/Ivenkova/lab4.1#

```

Рис. 1 — Результат со стороны сервера

```

Подключено к SSH-серверу. Введите команды (например, mkdir, removedir, list, move, remove, ping). Введите 'exit' для выхода.
Введите команду: list
Результат:
  abc1.txt
  go.mod
  go.sum
  server.go
  test1

Введите команду: mkdir testtest
Результат:
  Директория создана

Введите команду: list
Результат:
  abc1.txt
  go.mod
  go.sum
  server.go
  test1
  testtest

Введите команду: removedir test1
Результат:
  Директория удалена

Введите команду: list
Результат:
  abc1.txt
  go.mod
  go.sum
  server.go
  testtest

Введите команду: move abc1.txt testtest/abc.txt
Результат:
  Файл перемещен

Введите команду: list
Результат:
  go.mod
  go.sum
  server.go
  testtest

Введите команду: list testtest
Результат:
  abc.txt

```

Рис. 2 — Результат со стороны клиента 1

```
Введите команду: move testtest/abc.txt abc2.txt
Результат:
Ошибка перемещения файла: rename testtest/abc.txt abc2.txt: no such file or directory

Введите команду: move testtest/abc.txt abc2.txt
Результат:
Файл перемещен

Введите команду: list
Результат:
abc2.txt
go.mod
go.sum
server.go
testtest

Введите команду: ping google.com
Результат:
PING google.com (142.250.74.110) 56(84) bytes of data.
64 bytes from arn11s10-in-f14.1e100.net (142.250.74.110): icmp_seq=1 ttl=117 time=20.1 ms
64 bytes from arn11s10-in-f14.1e100.net (142.250.74.110): icmp_seq=2 ttl=117 time=20.1 ms
64 bytes from arn11s10-in-f14.1e100.net (142.250.74.110): icmp_seq=3 ttl=117 time=20.0 ms
64 bytes from arn11s10-in-f14.1e100.net (142.250.74.110): icmp_seq=4 ttl=117 time=20.0 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 20.009/20.056/20.102/0.038 ms

Введите команду: exit
Отключение от сервера...
root@net4:~/ivenkova/lab4.1#
```

Рис. 3 — Результат со стороны клиента 2